

Unit testing with xUnit practical

Charaka Palansuriya, Florian Scharinger, Craig Thomson, Adrian Mouat and Mike Jackson

14 July 2016

1 Introduction

In this practical you will use the xUnit framework to develop unit tests. You can do the exercise in Java or Python. You will need to download some code for these practicals. This code is available through the LEARN area for this course. To access these files:

1. Log-in to LEARN at <http://www.learn.ed.ac.uk> OR via <http://www.myed.ed.ac.uk>, using your EASE username and password.
2. Click My Courses.
3. Click Programming Skills(2016-2017)[SS1-SEM1].
4. Click Labs.
5. Under Unit tests, click UnitTests.
6. Right-click on unittest.zip and select Save As...

Copy the code to your desktop or `ph-cplab.ph.ed.ac.uk` and decompress it. This should produce a directory called `unittest`.

2 DNA sequences

We will be developing code and tests that implement operations on DNA sequences. This example is inspired originally by the Python DNA function of Gordon Webster, the Digital Biologist (<http://www.digitalbiologist.com/2011/04/code-tutorial-getting-started-with-python.html>).

A DNA sequence is a chain of one or more organic molecules called nucleotides. Each nucleotide - adenine, cytosine, guanine, thymine - is represented by a letter - A, C, G and T. A DNA sequence is a chain of these e.g. AAAGTCTGAC.

Each of these nucleotides has a molecular weight:

- Adenine (A): 131.2
- Cytosine (C): 289.2
- Guanine (G): 329.2
- Thymine (T): 304.2

The molecular weight of a DNA sequence is the sum of the molecular weights of each of its nucleotides. For example, the molecular weight of GATTACCA is:

- A: $131.2 * 3 = 393.6$
- C: $289.2 * 2 = 578.4$
- G: $329.2 * 1 = 329.2$
- T: $304.2 * 2 = 608.4$
- Molecular weight: 1909.6

3 Code and unit tests

A `Sequence` class, that represents DNA sequences, and operations on these, and a related unit test class, are both provided.

3.1 Java

Java code is in the `Java` directory.

`src/dna/Sequence.java` contains a Java implementation of `Sequence`.

`test/dna/SequenceTest.java` contains a unit test class `SequenceTest`, written using the JUnit (<http://junit.org>) test framework.

The JUnit 4.10 JAR file, `junit-4.10.jar` is also included.

To compile the code, run:

```
$ javac -classpath .:src src/dna/Sequence.java
$ javac -classpath .:src:junit-4.10.jar test/dna/SequenceTest.java
```

To run the unit tests, using JUnit's test runner, run:

```
$ java -classpath .:src:test:junit-4.10.jar \
    org.junit.runner.JUnitCore dna.SequenceTest
```

```
JUnit version 4.10
```

```
...
```

```
Time: 0.02
```

```
OK (3 tests)
```

In what follows, we'll assume you are using the command-line. However, you could import, compile and run this code from within Eclipse and benefit from the graphical test visualisation features provided by Eclipse's support for JUnit.

3.2 Python

Python code is in the `Python` directory.

`src/sequence.py` contains a Python implementation of `Sequence`.

`test/test_sequence.py` contains a unit test class, `SequenceTestCase`, written using Python's `unittest` (<https://docs.python.org/2/library/unittest.html>) framework.

To run the tests, using Python's `nosetests` (<https://nose.readthedocs.org/>) test runner:

```
$ nosetests test/test_sequence.py
```

```
...
```

```
-----
```

```
Ran 3 tests in 0.0025s
```

```
OK
```

To run the tests, and also generate an xUnit XML test report:

```
$ nosetests --with-xunit test/test_sequence.py
```

```
...
```

```
-----
```

```
Ran 3 tests in 0.005s
```

```
OK
```

```
$ cat nosetests.xml
```

4 Look at the code and tests

Look at the code and existing for your language of choice. The tests implemented are *correctness tests*, they test the function behaves correctly when given valid inputs.

Note the use of *assertions*, an integral part of xUnit test frameworks such as JUnit or nosetests. Assertions:

- Compare actual against expected results.
- Cause the test to pass or fail.
- Encapsulate common checks for convenience.
- Take the form of methods which test arguments for equality or inequality or check whether Boolean (true or false) conditions hold.

All tests must assert something! Tests without assertions don't test anything.

Note also the naming of the functions and classes. This is intentional. It makes it easier for a developer to recognise that these are tests and what parts of the code they test. Additionally, JUnit and nosetests expect tests to be named in this way so they can automatically find the tests and run them, without you specifically having to tell them the names of every test you have implemented.

5 Implement unit tests for existing methods

Add a unit test that tests if a sequence is created with a lower-case sequence (e.g. `gattacca`) returns an upper-case sequence via `getNucleodites` (in Java) or `nucleotides` (in Python).

Add a unit test that tests that if the sequence is an empty string then the molecular weight of the sequence is 0.0. This is an example of a *boundary test* (https://en.wikipedia.org/wiki/Boundary_testing) as, up to now, we might assume that the strings must be non-empty. There can be both valid and invalid boundary tests. For example, if testing the implementation of a sine function we could test what happens if it is given values 0 or 360 degrees which are valid boundaries, and what happens when it is given values of -1 or 361 which are invalid boundaries. For invalid boundaries, we might then want to add error handling code to our function to make it more robust. How our code responds to boundary conditions is specific to its domain and the problem it is intended to solve.

6 Implement a unit test with floating point numbers

Add a unit test that tests that if the sequence is `GATTACCA` then the molecular weight of the sequence is 1909.6.

In Java, use `assertEquals` e.g.

```
assertEquals("Weight returned was unexpected",
            1909.6,
            sequence.getWeight(),
            0.01);
```

In Python, use `assertAlmostEqual` e.g.

```
self.assertAlmostEqual(1909.6,
                        sequence.getWeight(),
                        delta=0.01,
                        msg="Weight returned was unexpected")
```

These test methods each have four arguments:

1. Informative message for if the test fails.
2. Expected value, the value we expect.
3. Actual value, the value from our method that we are testing for correctness.
4. A delta, 0.01.

The delta allows us to get round a problem of using floating point numbers. To see what this problem is, set the delta to be 0.0000000000001, then rerun the tests.

For Java you will see a failure like:

```
There was 1 failure:
```

```
1) testCalculateWeightGattacca(dna.SequenceTest)

java.lang.AssertionError: Weight returned was unexpected
expected:<1909.6> but was:<1909.6000000000001>
```

For Python you will see a failure like:

```
FAIL: test_calculate_weight_gattacca (test_sequence.SequenceTestCase)
-----

Traceback (most recent call last):

  File

"/home/mjj/prog-skills/PS/Labs/DNAnu/python/test/test_sequence.py",

  line 41, in test_calculate_weight_gattacca

    msg="Weight returned was unexpected")
AssertionError: Weight returned was unexpected
-----
```

This is because computers don't handle floating point numbers very well. As a result, we should never compare floating point numbers for exact equality, but only to equality within a given threshold. This is the role of the delta. Here, the numbers are considered equal if:

```
Math.abs(expected - actual) < delta
```

Put the delta back to 0.01 and rerun your tests. They should now pass again.

The choice of an appropriate delta depends very much on the nature of the domain for which the code is being written.

Implement a unit test to test that the molecular weight of ACGT is 1053.8.

7 Implement a unit test to detect errors

If you look at the `Sequence` constructor, it calls a validation function (`isValid` in Java and `is_valid` in Python) which checks that the sequence string only contains A, C, G, T (or their lower-case equivalents, a, c, g, t). If there is an unrecognised nucleotide (for example X), then Java will throw an `IllegalArgumentException` and Python will raise an `AssertionError`.

It is good practice to test how our code handles invalid or illegal arguments, not just valid and legal ones.

One name for these tests are *negative tests*. As another example, a negative test for a sine function in Python might be to test `sin("123")`. Negative tests can help to ensure that our code can gracefully handle invalid inputs, caused, perhaps, by a developer or by unexpected inputs from a user.

We can add tests for these as follows. For Java, we can call the constructor with an invalid sequence, and wrap this call in a `try-catch` block and if the expected exception is not thrown, then record the test as having failed. For example:

```
@Test
public void testSequenceAcgtx() {
    try {
        new Sequence("ACGTX");
        fail("Expected IllegalArgumentException");
    } catch (IllegalArgumentException e) {
        // Expected.
    }
}
```

For Python, the `unittest.TestCase` class provides a `assertRaises` function that allows us to check that an expected error is raised as follows:

```
def test_sequence_acgtx(self):
    with self.assertRaises(AssertionError):
        Sequence("ACGTX")
```

8 Implement a new method and unit tests

Add a `complement` method to `Sequence`. Given a DNA sequence consisting of A, C, T and G, we can create its complement, cDNA, by applying a mapping to each nucleotide in turn:

- A => T
- C => G
- T => A
- G => C

For example, given DNA strand GTCA, the cDNA is CAGT. Once you've added `complement`, add some unit tests.

Test Driven Development (https://en.wikipedia.org/wiki/Test-driven_development) is an approach to writing code where you write the tests first (based upon what methods or functions are intended to do), then implement the methods or functions so that the tests pass. The approach is as follows:

1. Quickly add a test.
2. Run all tests and see the new one fail.
3. Make a little change.
4. Run all tests and see them all succeed.
5. Refactor to remove duplication.

For more on test-driven development, see Beck, Kent (2002) Test-Driven Development, Addison Wesley, 2002. ISBN-10: 0321146530. ISBN-13: 978-032114653. <https://www.amazon.co.uk/Test-Driven-Development-Addison-Wesley-Signature/dp/0321146530>.

The antiparallel of a DNA sequence is determined by creating the complement of the sequence and then reversing it. The antiparallel of GTCA is TGAC.

Think of some tests to test a new `antiparallel` method for `Sequence` that would implement the antiparallel of a sequence.

Now, write a test. Then implement just enough of `antiparallel` so that the test passes.

Now, write the next test, then change and/or extend `antiparallel` so that this test passes too.

9 Conclusion

Once you've completed the practical, consider how the use of the test framework changed your development experience. In such a small example the overhead of unit testing may seem an unnecessary burden, but, hopefully, you should be able to see how it can quickly pay for itself and actually result in faster development times as you can then refactor your code with some comfort that if you introduce a bug, it could be picked up by your tests. Even with this small example there's a good chance you didn't get it quite right the first time and the tests should have caught this.

Consider the advantages and disadvantages of the test-driven development model; did you enjoy it? Can you see how many developers find it helps them to break down large problems into manageable chunks and have confidence in their changes?

10 Find out more...

Check out the unit testing guidelines: Geotechnical Software Services (2010). Unit Testing Guidelines. Version 4.1, December 2010. <http://geosoft.no/development/unittesting.html>

The Software Sustainability Institute's build and test examples, (https://github.com/softwareaved/build_and_test_examples) includes sample unit tests, in xUnit test frameworks, for popular programming languages.